

Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Monterrey



TE3001B

Fundamentación de robótica

Robotics Control Lab 4.2

Alumnos:

Josue Ureña Valencia	IRS A01738940
César Arellano Arellano	IRS A00839373
Jose Eduardo Sanchez Martinez	IRS A01738476
Rafael André Gamiz Salazar	IRS A00838280

Docente:

Nezih Nieto Gutiérrez

Fecha de entrega:

24 de febrero del 2026

Índice

Contexto.....	3
Generador de trayectoria.....	3
Controlador Proporcional-Derivativo (PD).....	4
Publicación del comando cartesiano.....	4
Inyección de Perturbaciones.....	5
Perturbación senoidal (Determinística).....	5
Perturbación gaussiana (Estocástica).....	5
Resultados.....	6
Error en la medición.....	12

Contexto

El sistema desarrollado implementa un controlador cartesiano de posición para el *xArm Lite 6*, utilizando *control* en tiempo real mediante *MoveIt* dentro del entorno *ROS 2*.

El objetivo principal es lograr que el efector final siga una trayectoria, para después evaluar el desempeño del controlador implementado bajo condiciones nominales y perturbaciones externas *determinísticas* y *estocásticas*.

El sistema se compone de tres bloques funcionales:

- Se genera una trayectoria cartesiana
- Se establece un controlador PD en espacio cartesiano
- Se inyectan las perturbaciones

Cada uno cumple un papel específico dentro de la arquitectura.

Generador de trayectoria

La trayectoria se define directamente en coordenadas cartesianas, el sistema genera por sí mismo referencias en términos de posición x_d , y_d , z_d dentro del marco `link_base`.

En cada ciclo de control se obtiene la posición actual del efector final mediante TF, resolviendo la transformación entre `link_base` y `link_eef`.

```
Python
trans = self.tf_buffer.lookup_transform(
    "link_base", "link_eef", rclpy.time.Time()
)

current = np.array([
    trans.transform.translation.x,
    trans.transform.translation.y,
    trans.transform.translation.z
])
```

Con esta información, el error cartesiano se calcula como:

```
Python
error = target_pos - current
```

Este vector representa la diferencia espacial directa entre la posición deseada y la real en coordenadas XYZ.

Controlador Proporcional-Derivativo (PD)

El controlador implementado es de tipo Proporcional-Derivativo:

$$v(t) = K_p e(t) + K_d \dot{e}(t)$$

El término proporcional genera una velocidad en la dirección del error, empujando al sistema hacia la referencia. El término derivativo introduce amortiguamiento dinámico, reduciendo el sobre impulso y mejorando la estabilidad transitoria:

```
Python
d_error = (error - self.prev_error) / dt
v = self.kp * error + self.kd * d_error
```

- self.kp corresponde al término proporcional
- self.kd corresponde al término derivativo
- dt es el tiempo entre iteraciones

Publicación del comando cartesiano

A partir del error, se genera un comando de velocidad lineal en forma de mensaje TwistStamped, el cual especifica velocidades cartesianas expresadas en el sistema de referencia de la base del robot, el cual contiene componentes lineales v_x , v_y , v_z .

```
Python
cmd = TwistStamped()
cmd.header.frame_id = "link_base"

cmd.twist.linear.x = float(v_xyz[0])
cmd.twist.linear.y = float(v_xyz[1])
cmd.twist.linear.z = float(v_xyz[2])
```

El campo frame_id se establece como link_base, garantizando que las velocidades estén expresadas respecto al marco de la base del robot, manteniendo coherencia con el cálculo del error. MoveIt transforma internamente dichas velocidades cartesianas en velocidades articulares mediante el Jacobiano del manipulador:

$$\dot{\theta} = J^{-1}v$$

Inyección de Perturbaciones

Perturbación senoidal (*Determinística*)

Se añade una señal periódica:

$$v_p(t) = A \sin(2\pi ft)$$

Esta perturbación permite analizar la respuesta del sistema ante perturbaciones armónicas, observando posibles desfases y variaciones en el error. En el nodo de perturbación, la señal senoidal se genera como:

```
Python
s = math.sin(2.0 * math.pi * self.sine_freq_hz * (time.time() - self.t0))
dp[axis] = self.sine_amp_linear * s
```

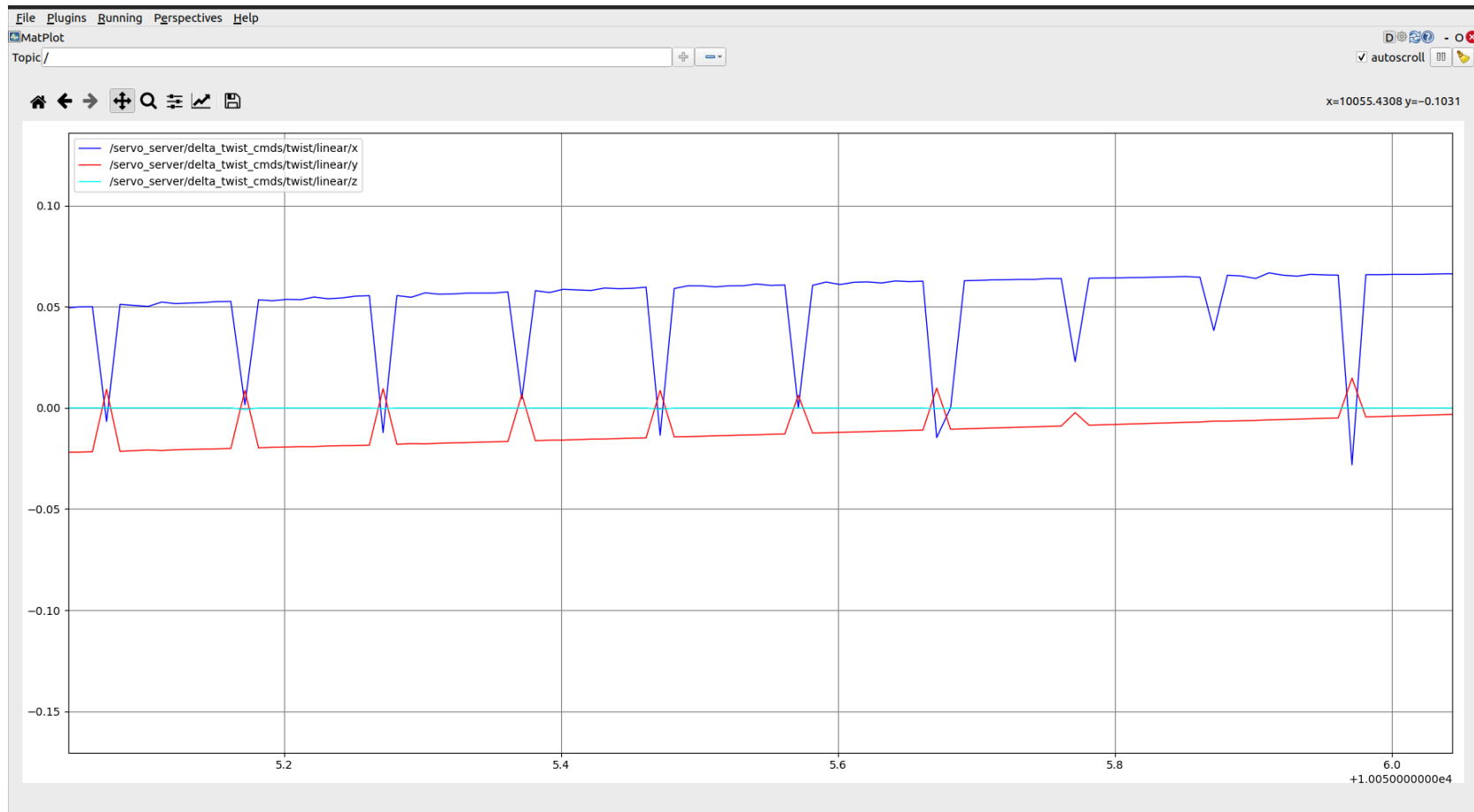
Perturbación gaussiana (*Estocástica*)

El ruido gaussiano se implementa como:

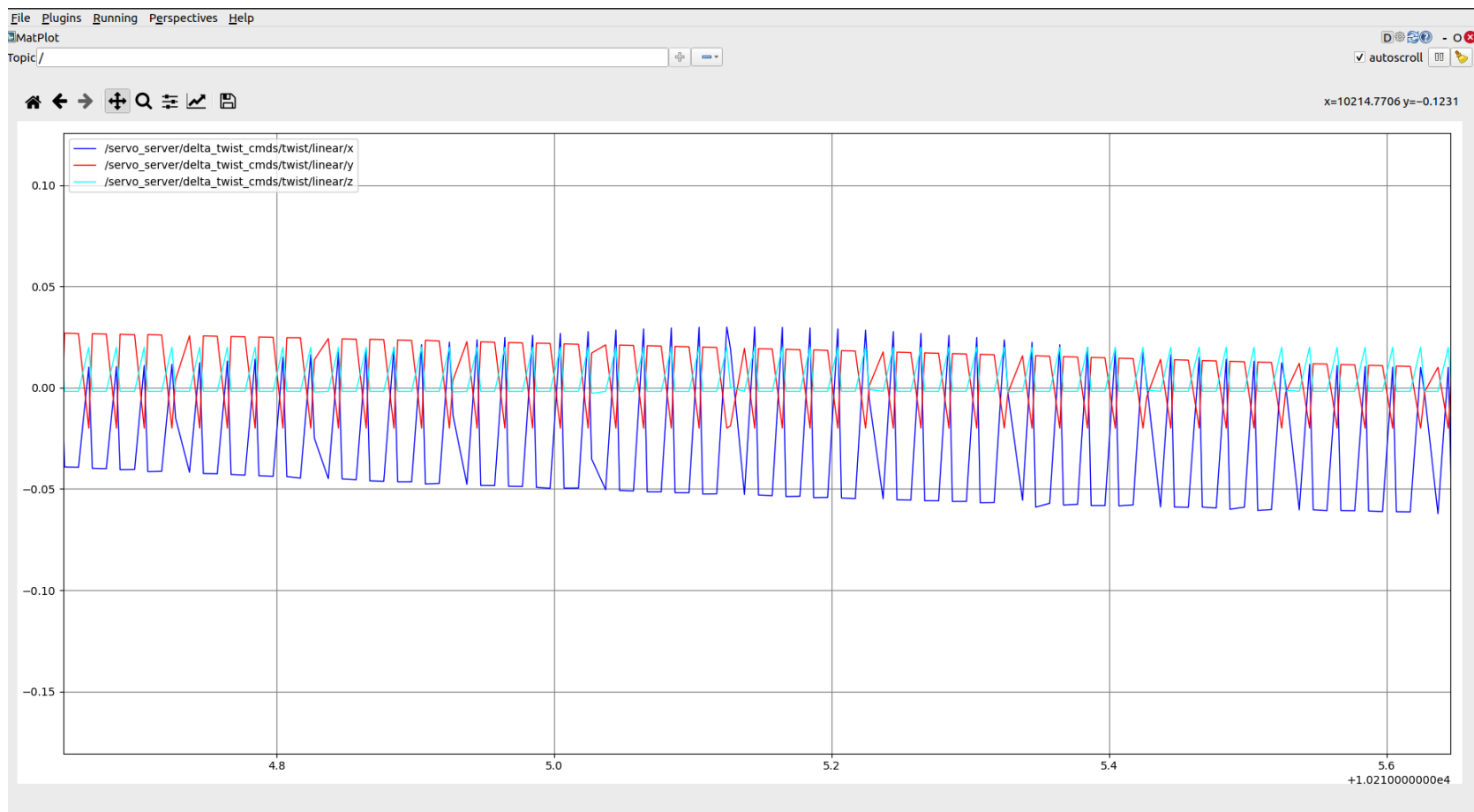
```
Python
return self.rng.normal(0.0, self.noise_std_linear, size=3)
```

Este evalúa la sensibilidad del controlador ante señales impredecibles de alta frecuencia. Es importante notar que el término derivativo tiende a amplificar ruido de alta frecuencia, lo cual impacta directamente en la magnitud de la señal de velocidad generada.

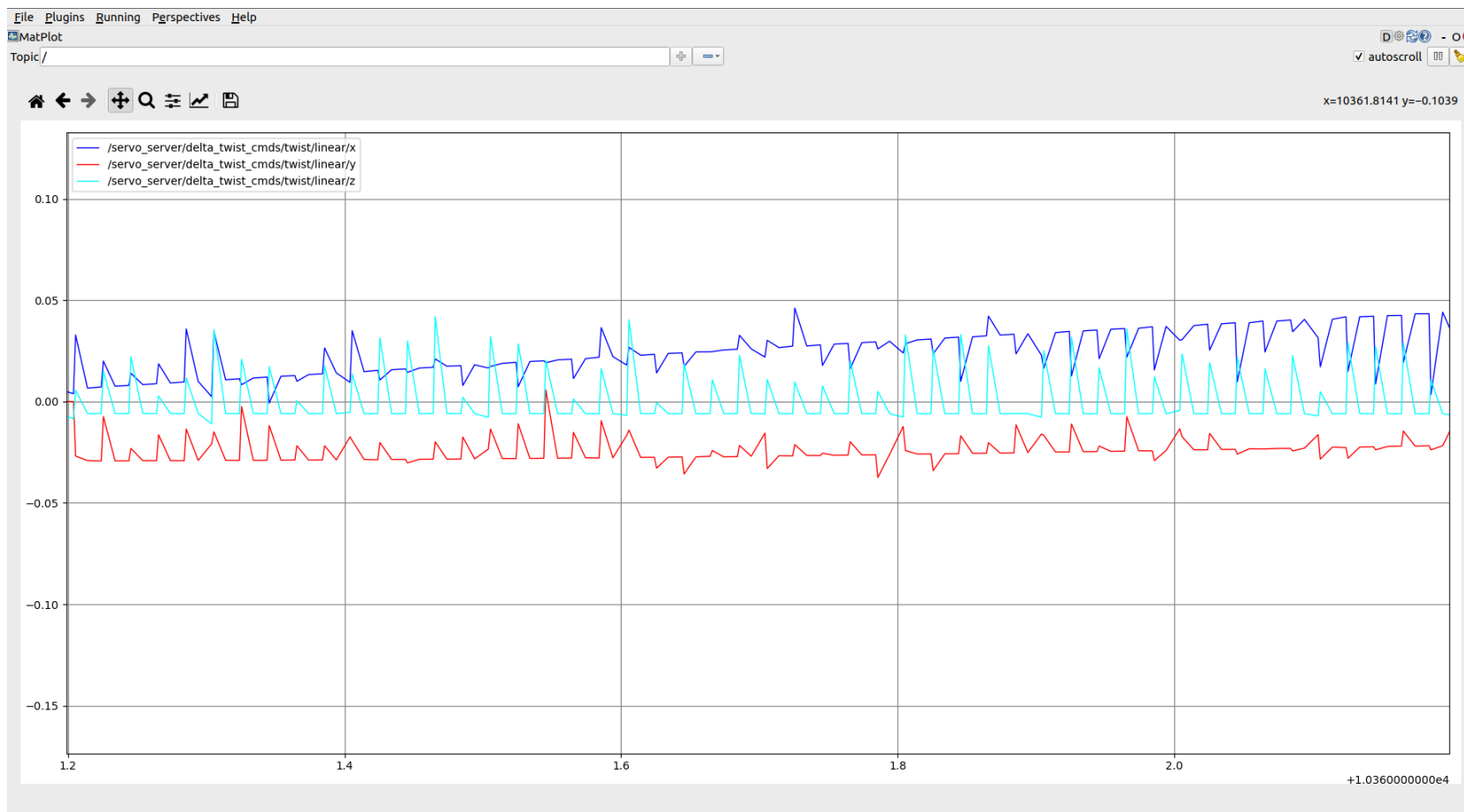
Resultados



El objetivo aquí es mostrar el comportamiento base del sistema antes de introducir perturbaciones.

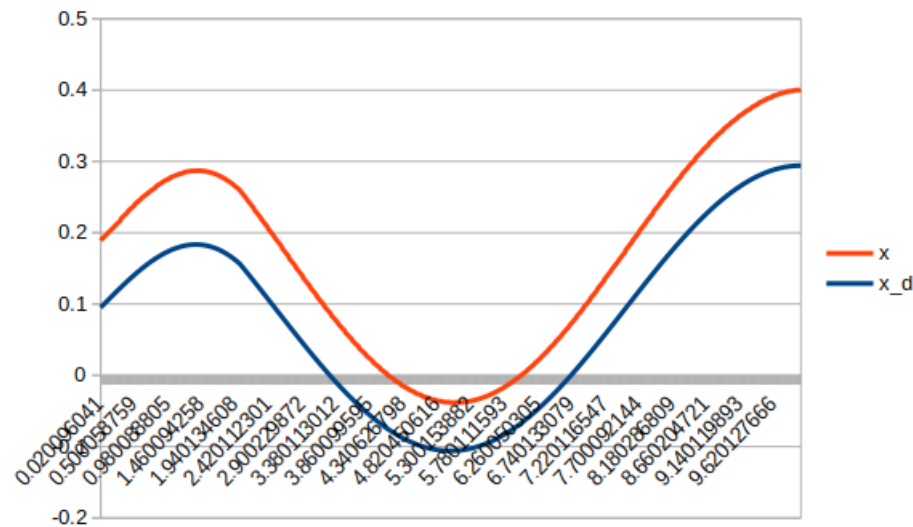


Acá se puede analizar cómo responde el sistema ante perturbaciones senoidales.

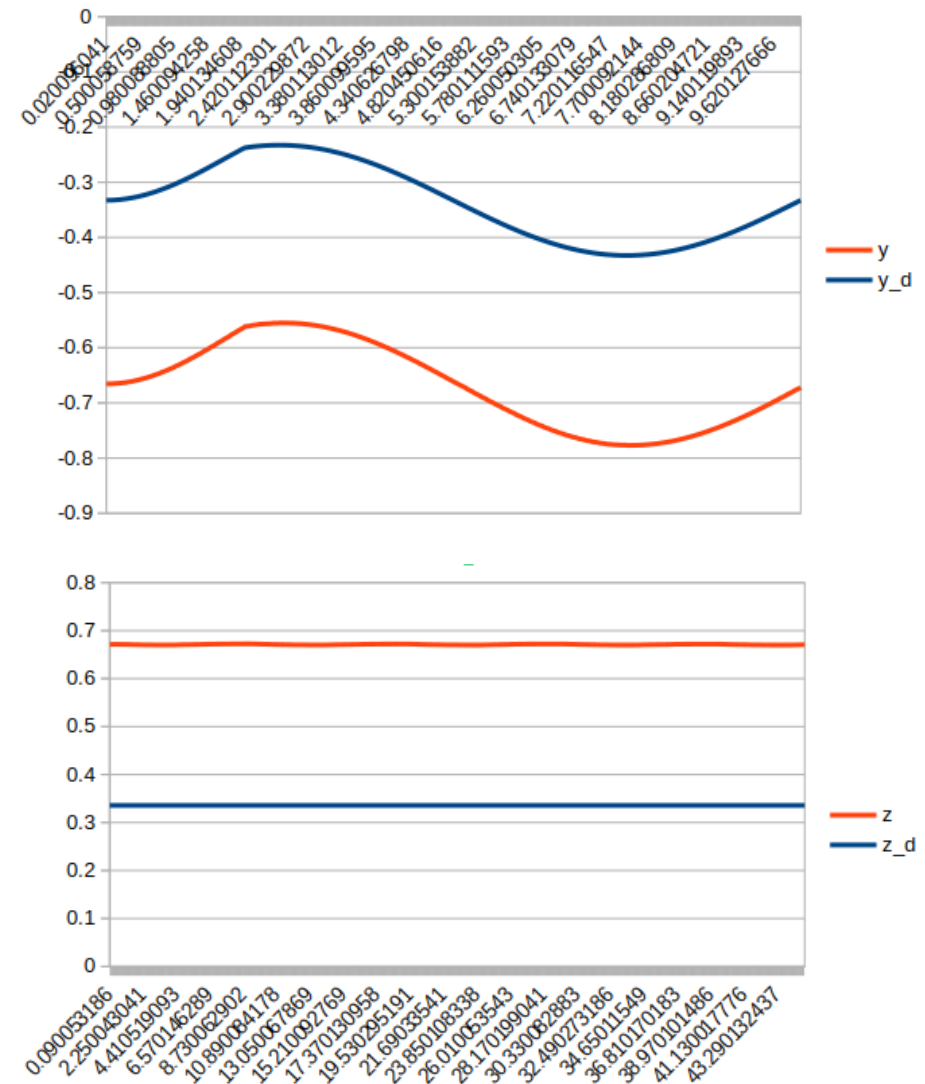


Estas gráficas muestran mayor irregularidad en las señales, esto cuando se presentan perturbaciones gaussianas.

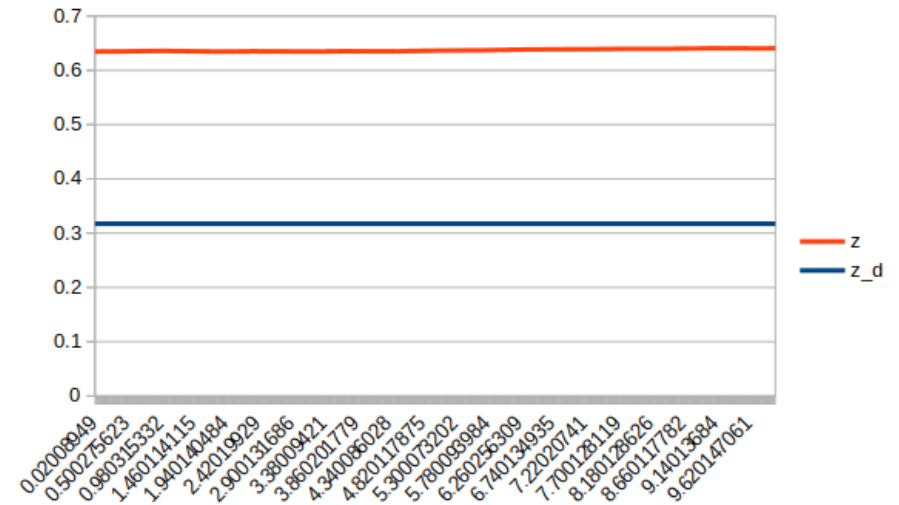
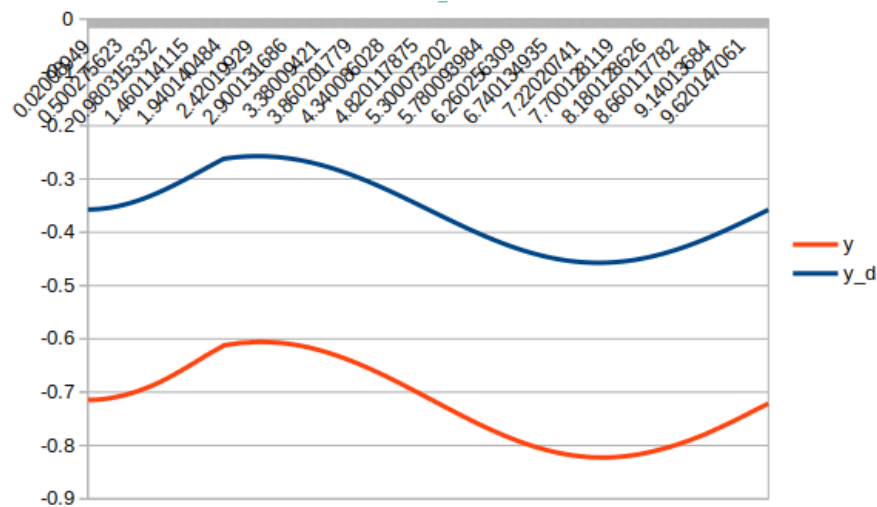
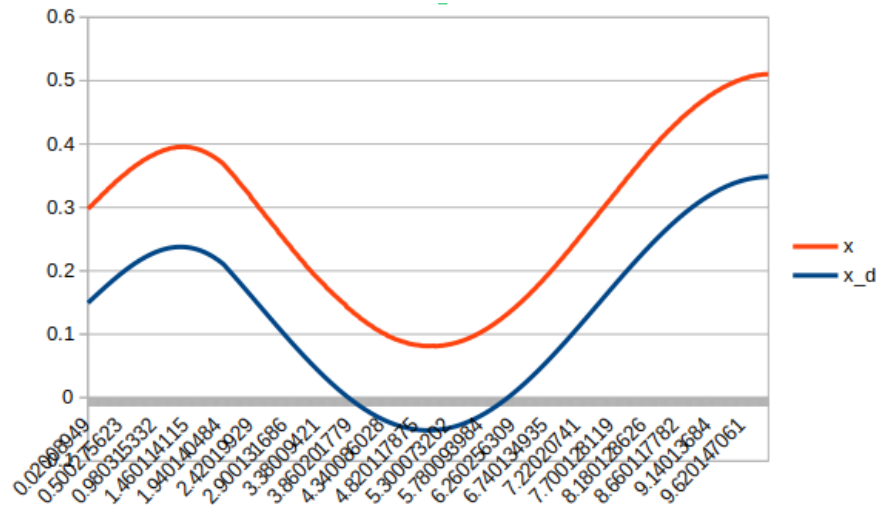
Gráficas generadas de posición real y deseada bajo condiciones nominales a través del tiempo



Se muestra que el controlador cartesiano PD logra un seguimiento estable y amortiguado de la trayectoria deseada, también observamos un error, un desfase entre la señal deseada y la señal real, especialmente en los cambios de pendiente máxima, este desfase se debe a que el sistema en sí, tiene dinámica propia, la conversión cartesiana-articular mediante el Jacobiano introduce dinámica adicional y existe latencia de procesamiento en el lazo de control. Además durante la experimentación se pudo notar como si se realizaba la trayectoria correcta, solo que a escala de la trayectoria definida, como resultado se observa en las gráficas ese error entre la trayectoria deseada y la que realmente realiza.



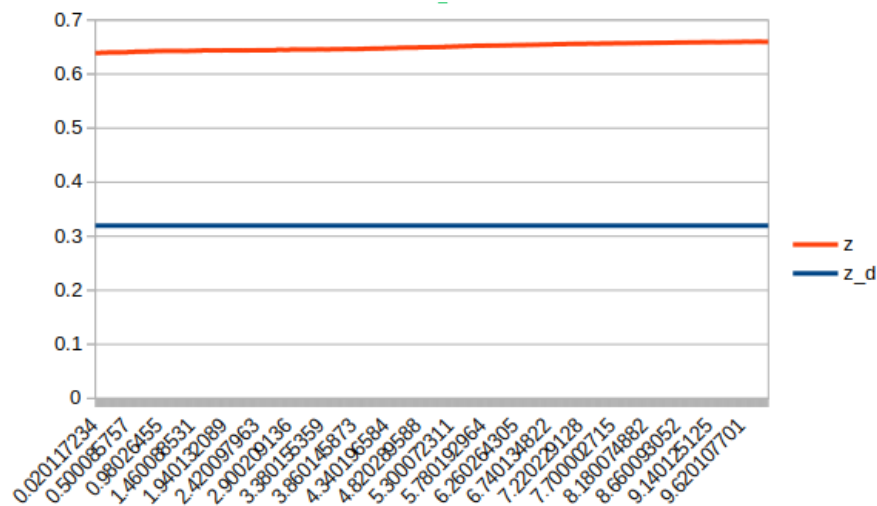
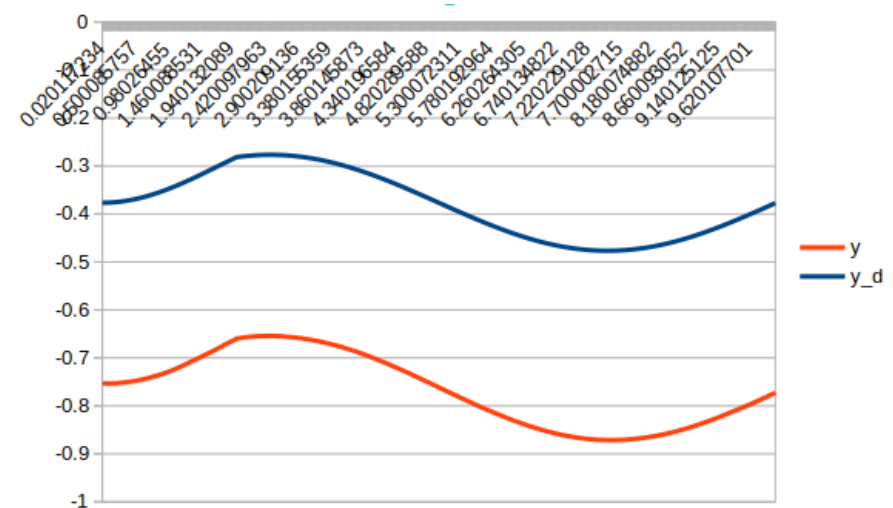
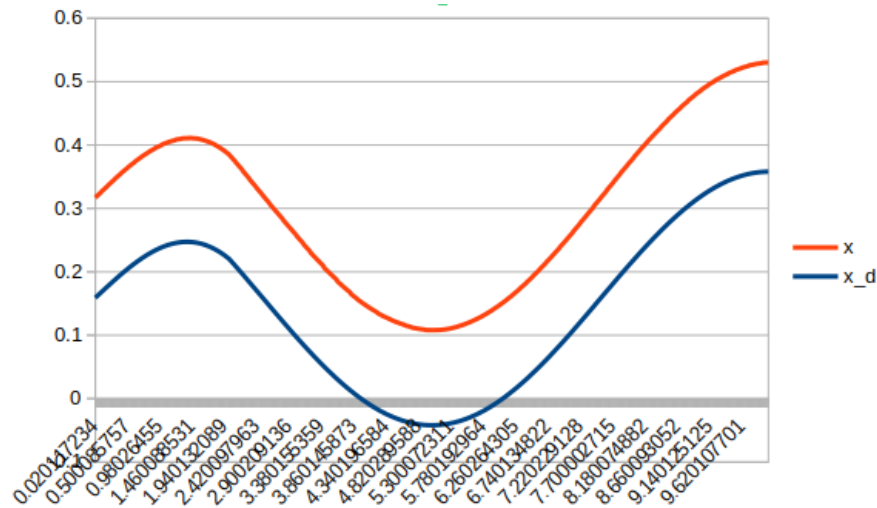
Gráficas generadas de posición real y deseada bajo condiciones perturbaciones determinísticas a través del tiempo



Ahora observamos mayor separación entre señal deseada y señal real, la amplitud del error incrementada, se nota una ondulación más marcada, esto ocurre porque el controlador debe compensar simultáneamente:

- La dinámica propia del sistema
- La trayectoria deseada
- La señal periódica adicional
-

Gráficas generadas de posición real y deseada bajo condiciones perturbaciones estocásticas a través del tiempo



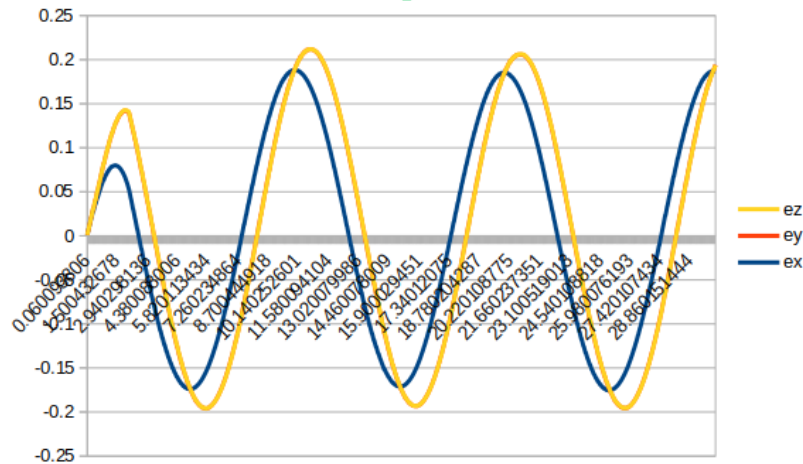
Aquí la perturbación no es periódica ni frecuente, lo que implica que el sistema debe reaccionar ante variaciones impredecibles de distinta magnitud y frecuencia.

A pesar del incremento en la variabilidad, podemos notar que el error permanece adecuado, no hay gran crecimiento acumulativo y no se observa inestabilidad.

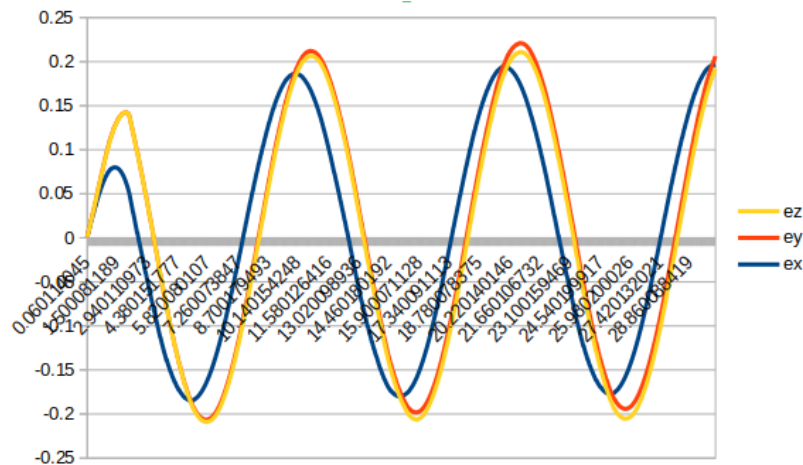
Lo cual indica que el control posee robustez frente a perturbaciones

Error en la medición

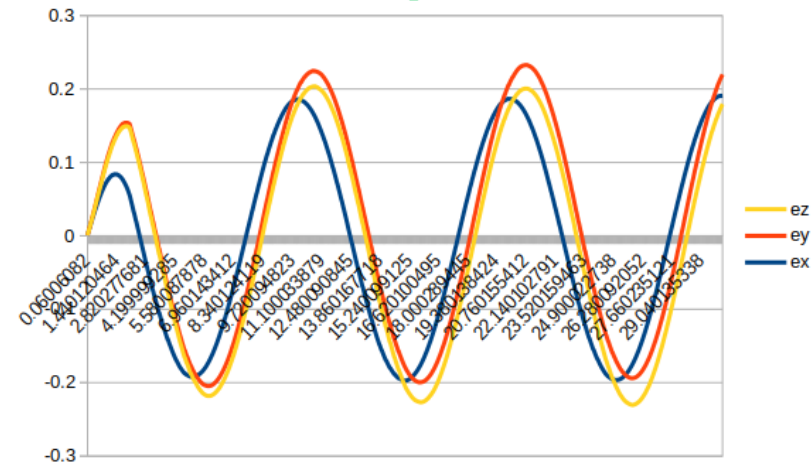
Nominal



Senoidal



Gaussiana



Observando los errores, podemos confirmar visualmente lo concluido en las páginas anteriores. Aunque el desempeño disminuye conforme aumenta el grado de la perturbación, el controlador PD mantiene estabilidad y capacidad de seguimiento.

Además, durante el experimento, se observó que a pesar de que la ejecución de la trayectoria sin perturbaciones ya presentaba errores, dichos errores se mantuvieron muy similares durante las perturbaciones senoidales y gaussianas, demostrando que nuestro controlador logró mitigar las perturbaciones agregadas.

Conclusión

En condiciones nominales, el controlador opera dentro de un margen estable donde la acción proporcional corrige el error estacionario y la acción derivativa introduce amortiguamiento adecuado sin amplificar señales indeseadas.

Al introducir una perturbación senoidal, el sistema continúa siendo estable, pero se observa un incremento en la amplitud del error y un ligero desfase, este comportamiento corresponde a la respuesta a un sistema en lazo cerrado ante una perturbación.

Finalmente, bajo la perturbación gaussiana, el sistema enfrenta el escenario más dinámico. La señal real conserva la forma general de la referencia, pero presenta variaciones rápidas e irregulares, este fenómeno se explica por el término derivativo ante componentes de alta frecuencia presentes en el ruido. A pesar de ello el sistema mantiene estabilidad, lo que demuestra robustez de control.

En conjunto, los resultados observados validan la implementación del controlador PD y demuestran su capacidad para mantener estabilidad y desempeño aceptable incluso bajo condiciones críticas.

Github repository: https://github.com/Jose05M/xarm_lite6_lab4.2/tree/main

